RFC: Learning Control Plane (Agent Self-Improvement)

RFC: Learning Control Plane (Agent Self-Improvement)

- **Status:** Draft v0.9
- **Date:** 2026-07-08
- **Target:** 3.12+ (lands after the trace-derived evals stack, PR #115)
- **Owners:** Planner / Platform
- **Related:** RFC_TRACE_DERIVED_DATASETS_AND_EVALS, RFC_SKILLS_LEARNING_V213, skills.md, auto-seq.md

1. Summary

Make PenguiFlow agents **improve over time from real interactions** without redeploying agent code, at enterprise quality. Agents learn **runtime assets**, never self-edit core code. v1 learns two asset types:

1. **Learned skills** — reusable playbooks injected into the planner context (advisory; the LLM can ignore them). Low blast radius.
2. **Learned auto-seq edges** — typed, scoped `from_tool → to_tool` transitions that let the planner skip the LLM. **Design principle: auto-learning implies auto-seq** — a promoted edge grants auto-seq for its transition directly; the developer's static `auto_seq_execute` flag is no longer a precondition (the validation gate is the earned equivalent). Model-bypassing execution, so high blast radius — the grant is bounded by fire-time correctness, opt-out, a read-only default, and reversibility (§5.7).

The two assets share **one signal spine and one validation gate** but have **asymmetric promotion bars**: skills promote on a light gate; edges traverse shadow (divergence + fire-time safety) → canary (observed cohort outcome vs a **randomized ε-control**, §5.7) → active, and default to read-only + human approval. Edge-outcome evidence enters only at canary, the first stage where the edge actually fires.

The capability is a **Learning Control Plane**: a separate, scheduled (offline) service that mines, validates, and promotes assets. PenguiFlow **core** grows a bounded set of *net-new* primitives (contract types, a learned-skill write path, a patch-point registry + planner edge application, and a global kill-switch). Nothing runs in the request path except activation of already-promoted assets.

Skill *injection/formatting* (`format_for_injection`) and the auto-seq *detection scaffolding* (the `DetectionResult` type and auto-seq event types) are reused as-is. The skill *write path*, a **scope-aware read/dedup path** (the existing name-keyed read collapses same-named scoped rows by fetch order, not by scope precedence — net-new work), a learned-store provider, tenant/window trace enumeration, full-live-catalog ambiguity scanning, and the entire edge-application path are **net-new core work** (see §8).

2. Motivation

We run several production agents on PenguiFlow. We already own both halves of the learning signal, which most teams must build from scratch:

- **Execution truth** — the `StateStore` (`Trajectory`, `PlannerEvent`, flow events): *what the agent did*.
- **Outcome truth** — the Iceberg memory server (`interaction_fragment.feedback`, `knowledge_fragment.contested`, `preference_score`): *whether it was good*.

And we have a **validation substrate** landing via PR #115 (`penguiflow/evals/`): trace → dataset → metric → baseline-vs-candidate sweep with a val/test holdout regression gate.

The missing pieces are (a) a join between the two signal sources, (b) compiling outcome truth into eval labels **without leaking that signal into the gate**, (c) candidate generation with statistical gating, and (d) safe promotion machinery. This RFC specifies them.

3. Goals / Non-goals

Goals - Learn skills and auto-seq edges from production traces + Iceberg feedback. - Validate every candidate offline against trace-derived datasets before it can affect any user, with **statistical** (not point-score) gating and an **independent** authority signal. - Promote each asset class on its own ladder, with scope, TTL/decay, rollback, and a **global kill-switch**: skills go draft → active on a light gate (structural validation + redaction + dedup + the holdout regression check) once they beat baseline on held-out runs; edges go draft → shadow → canary → active, with **outcome** evidence entering only at canary — the first stage where the edge actually fires — measured against a randomized ε-control (§5.7, §16). - Keep the production request path unchanged except for activating already-promoted assets. - Reuse existing seams where they genuinely exist (§8).

Non-goals (v1) - No self-editing of core code, tool policy, auth scopes, or flow topology. - No autonomous (no-human) activation of **write-capable** auto-seq edges. - No arbitrary DAG/recipe synthesis. “Recipes” in v1 = typed single-hop auto-seq edges only; consecutive learned hops are capped (default 1) so the planner never runs an unvalidated multi-hop chain model-free (§5.7). - No online/in-request learning. The loop is a scheduled batch job. - No new optimizer (DSPy/GEPA). v1 uses manual-sweep candidate evaluation; GEPA can slot in later at candidate generation (the metric signature is already GEPA-compatible).

4. Terminology, the join key, and tenancy

Term	Meaning
`trace_id`	A single planner run (≈ one agent turn). Primary key of `Trajectory` / `PlannerEvent`.
`run_id`	Iceberg’s equivalent of `trace_id` (platform-injected, 1:1).
Join key	`StateStore pf_trajectories.trace_id == Iceberg interaction_fragment.agent_interaction_id` (value mapping architect-confirmed, 1:1). The join is **Iceberg-driven**: the spine enumerates `interaction_fragment` rows for a `(tenant, window)` — each already carries `tenant_id` / `user_id` / `agent_interaction_id` — as a batch set-extract, then fetches each StateStore `Trajectory` by `trace_id`. Tenancy on the join therefore always comes from the Iceberg side, never from the trace side.

Join cost is deployment-dependent — verify the target environment. Reading `agent_interaction_id` off `interaction_fragment` depends on the column existing. The Iceberg store probes for it at query time (`_ensure_agent_interaction_id_capability`, `sql_stores.py:1473`). Where the column exists — the current schema ships it with the `idx_interaction_agent_id_unique` unique index — the set-extract is indexed. Where it is absent, the store falls back to `metadata ->> 'agent_interaction_id'` (`sql_stores.py:1972`), an **unindexed** scan on Postgres and unresolvable on SQLite. No Iceberg schema migration is required *on current-schema deployments*; before enabling mining the spine asserts the target carries the indexed column at startup, and if it is on the metadata-fallback path the spine backfills the column and adds the index first. Driving the join as one batch set-extract per `(tenant, window)` — rather than N per-trace point lookups — keeps the nightly job’s cost bounded as trace volume grows.

Two tenancy sources of truth — reconcile explicitly. StateStore's `Trajectory` / `PlannerEvent` models do not carry `tenant_id` / `user_id` as columns (the auxiliary `RemoteBinding` carries raw `tenant_id` / `user_id`, but it is not present on every trace and is not the mining unit). Tenancy is authoritative on the **Iceberg** side of the join (`tenant_id` / `user_id` on every `interaction_fragment` row) and drives **all offline mining and labels** — and because the join is Iceberg-driven (§4 join key), mining never depends on the trace side carrying tenancy at all. Request time is different: the skills scope filter reads `tool_context.get("tenant_id")` (`skills/provider.py:96`), but `tool_context` is **not durably persisted**. `Trajectory.serialise()` JSON-round-trips `tool_context` and sets it to `None` whenever *any* value is not JSON-serializable (`planner/trajectory.py:245–251`); `tool_context` can hold live service objects, so a single non-JSON value nulls the **entire** persisted `tool_context` — it is not a dependable tenancy source.

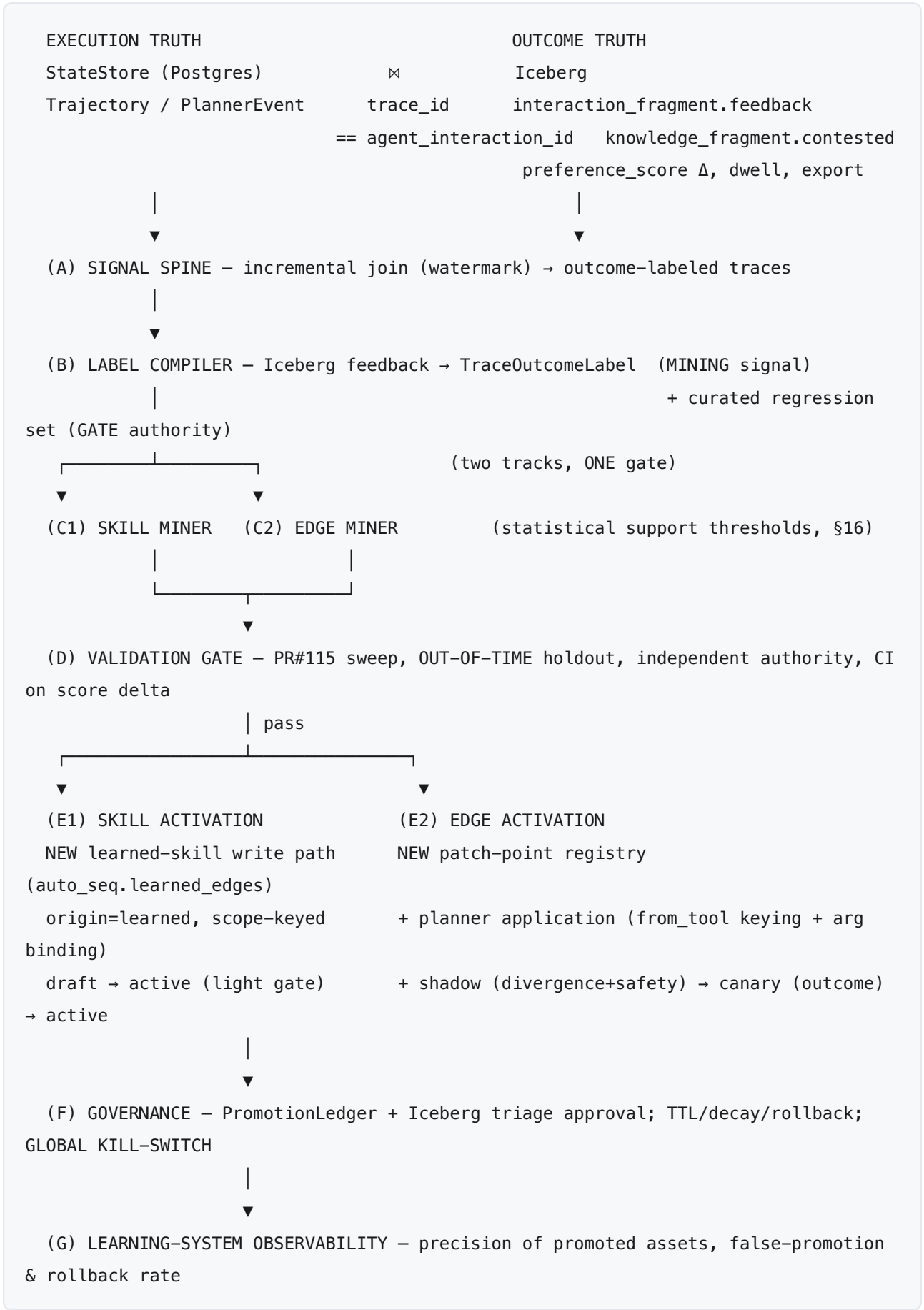
v1 therefore persists a **first-class, sanitized scope snapshot** on the trajectory, captured at request start independently of the full `tool_context`:

```
scope_snapshot = {
    "scope_mode": "tenant" | "project" | "global",
    "tenant_id": str | None,
    "project_id": str | None,
    "user_id": str | None,   # pseudonymized per §10
}
```

It carries only the scalar tenancy fields lifted from `tool_context` at run start, sanitized through the §10 redaction profile, and is serialised on its own field — never gated by whether the rest of `tool_context` is JSON-clean. This is net-new core work (§8): a typed `scope_snapshot` on `Trajectory` plus its serialise/deserialise.

v1 rule: **offline scoping uses Iceberg tenancy (authoritative for mining and labels); request-time scoping uses the persisted `scope_snapshot` .** A missing `scope_snapshot` is **not** grounds to discard a trace **when an Iceberg tenancy row exists**: mining and labeling proceed on Iceberg tenancy, and the snapshot is required only to admit a trace into the **request-time canary cohort** (§5.7). A trace with **neither** an Iceberg row **nor** a `scope_snapshot` has no tenant/project owner — `Trajectory` / `PlannerEvent` carry no tenancy (the auxiliary `RemoteBinding` is not the mining unit) — so it cannot be placed in any scope: it is **excluded from scoped mining** and may feed **only global execution-truth/regression** signals (§5.1). When both an Iceberg row and a snapshot are present the spine asserts they agree on `tenant_id` / `project_id` and drops only traces where those **disagree** — a genuine integrity violation. The snapshot's `user_id` is pseudonymized (§10) while the Iceberg `user_id` is raw, so `user_id` is **never** part of the agreement check; reconciliation is at the scope (tenant/project) level only. This keeps the two sources reconciled without the biased, silent data loss that depending on `tool_context` for request-time tenancy would cause.

5. Architecture


EXECUTION TRUTH
StateStore (Postgres)
Trajectory / PlannerEvent
⋈
trace_id
== agent_interaction_id
OUTCOME TRUTH
Iceberg
interaction_fragment.feedback
knowledge_fragment.contested
preference_score Δ, dwell, export
(A) SIGNAL SPINE — incremental join (watermark) → outcome-labeled traces
(B) LABEL COMPILER — Iceberg feedback → TraceOutcomeLabel (MINING signal)
+ curated regression set (GATE authority)
(two tracks, ONE gate)
(C1) SKILL MINER
(C2) EDGE MINER
(statistical support thresholds, §16)
(D) VALIDATION GATE — PR#115 sweep, OUT-OF-TIME holdout, independent authority, CI on score delta
pass
(E1) SKILL ACTIVATION
NEW learned-skill write path (auto_seq.learned_edges)
origin=learned, scope-keyed
draft → active (light gate)
(E2) EDGE ACTIVATION
NEW patch-point registry
+ planner application (from_tool keying + arg binding)
+ shadow (divergence+safety) → canary (outcome) → active
(F) GOVERNANCE — PromotionLedger + Iceberg triage approval; TTL/decay/rollback; GLOBAL KILL-SWITCH
(G) LEARNING-SYSTEM OBSERVABILITY — precision of promoted assets, false-promotion & rollback rate

Placement. Boxes **A, B, C, D, F (orchestration), G run in a separate control-plane service** (scheduled batch, nightly, aligned with Iceberg's consolidation). PenguiFlow **core** grows: the contract types (§9), the **learned-skill write path** (E1), the **patch-point registry + planner edge application** (E2), the **kill-switch**, and shadow/canary planner events.

5.1 (A) Signal spine (incremental)

Per scope/window: 1. Enumerate work since the last **watermark** (mirrors Iceberg's `knowledge_watermark` idempotency pattern), **driven from the Iceberg side**: pull `interaction_fragment` rows for the `(tenant, window)` — they carry `tenant_id` / `user_id` / `agent_interaction_id`, so tenant scoping lives on Iceberg, not the StateStore (whose `Trajectory` / `PlannerEvent` carry no `tenant_id`, §4). A net-new `SupportsTraceQuery` window enumeration backstops traces needing execution-truth labels without an Iceberg row (the existing `list_traces` requires a `session_id` and cannot do this; see §8). A backstopped trace carries no scope owner of its own: a trace with **neither an Iceberg row nor a `scope_snapshot`** cannot be placed in any tenant/project scope, so it is **excluded from scoped mining** and may contribute only to **global execution-truth/regression** signals. A scoped candidate is mined only from traces with a valid scope source — an Iceberg tenancy row or a present `scope_snapshot`. 2. Fetch `Trajectory` / `PlannerEvent` (`SupportsTrajectories.get_trajectory`, `SupportsPlannerEvents.list_planner_events`). 3. Reconcile tenancy (§4): label and mine on authoritative Iceberg tenancy, take request-time scope from the persisted `scope_snapshot`, and drop only on tenant/project disagreement — never on a missing snapshot. A trace with no scope source at all (no Iceberg row **and** no `scope_snapshot`) is excluded from scoped mining and feeds only global signals. 4. Output outcome-labeled traces. Read-only on both stores.

Incremental processing bounds cost: only new traces are scanned per run; full re-mining is an explicit, rare operation.

5.2 (B) Label compiler — and the leakage firewall

Compiles Iceberg feedback into one `TraceOutcomeLabel` per trace (§9.1). Mapping (initial):

Iceberg signal	Contribution
`feedback.intent == CORRECTION`; `knowledge_fragment.contested`	strong negative
`feedback.explicit_dislike`	negative
`feedback.explicit_like`; `feedback.exported`; `dwell_sec >= 0`	positive
`feedback.intent == COMMIT`; rising `preference_score`	strong positive
StateStore `PlannerEvent.error`, tool failures, replans	execution-negative

A positive label requires an Iceberg signal. Execution truth alone can only mark a trace **execution-negative**, never **good** — absence of an error is not success. A trace with **no Iceberg outcome row** is therefore labeled `unknown`: it is eligible for execution-truth/regression and **negative-skill** mining, but is **never counted as a "good" trace** for positive candidate mining (§5.3–5.4). This keeps a `scope_snapshot`-only trace (admitted for scoped mining in §5.1) from being mistaken for a success merely because it raised no error, and bounds the learner's positive bias to traces with real outcome evidence.

Leakage firewall (the central methodology point). The user-feedback signal is used to **mine and rank candidates**, so it must NOT be the gate authority. The firewall excludes the **mining signal** (Iceberg feedback) from the pass/fail decision — it does **not** exclude all outcome evaluation. Authority is asymmetric by asset class (§5.5): 1. **Skills — the positive signal is goal-success, not trajectory imitation.** A learned skill that reaches the goal a *better, different* way is correct, yet would fail metrics that reward reproducing the historical trajectory; with Iceberg labels held non-authoritative, imitation metrics alone would promote only candidates that look like the old plan. So skills are scored on an **outcome/goal-success signal** independent of the mining signal: an `evals/helpers.py` judge in **reference-guided** mode against a frozen, human-curated **gold set** (verifiable answer per task). This is a **required new capability on PR #115** — reference-guided scoring plus a paired per-item prediction output for §16 — *not* existing reuse: PR #115 ships an `llm_judge`, but the reference-guided/paired-output mode this gate depends on must land with it, and the methodology is gated on that API. The judge is a **calibrated signal feeding the §16 statistical bar**, never the final verdict on its own (LLM judges carry self-preference/position/verbosity bias). The imitation metrics (`trajectory_subset_match`, `extract_terminal_node`, `sequence_match`) are **demoted to regression tripwires**: a behavior change flips them, which *routes* the candidate to outcome validation — it never *blocks* the candidate for diverging. 2. **Edges — fidelity IS the authority.** A learned edge's contract is to reproduce a confirmed, deterministic transition, so the same imitation metrics are the **primary positive authority** for edges (with the §5.7 fire-time checks and the §5.7 shadow → canary ladder), backstopped by a cheap sample outcome check at the gate so an edge that faithfully reproduces a *suboptimal* historical transition is not certified on fidelity alone. Reproducing the known transition is the goal, not a bias. 3. **Independence is enforced, not assumed.** Gold labels are human-adjudicated **blind** to a candidate's mined signal, and any gold item is **firewalled out of the miner** for the cycle in which it serves as authority — so a trace that surfaced a candidate can never also be the gold that validates it. The judge model family differs from any candidate generator. This is what makes the firewall hold end-to-end. 4. **Out-of-time holdout.** Mine on window `W`; validate on a strictly later window `W+1`. A candidate must hold up on traces it was not derived from. 5. **Iceberg labels are a secondary, reported signal in the gate, never the sole pass/fail.** They may corroborate but cannot by themselves promote.

5.3 (C1) Skill miner

- Cluster **good** traces by normalized task signature; extract recurring successful sequences and failure-recovery sequences into `LearnedSkillCandidate` (§9.2); optional **negative skills**.

- Dedup via `source_hash`. Honor statistical support thresholds (§16).
- Persist only the **abstracted playbook** + provenance `trace_ids` — never raw observations or literal args (§10).

5.4 (C2) Edge miner

From outcome-labeled trajectories, propose single-hop `from_tool → to_tool` edges where: 1. the transition recurs across ≥ N **good** traces meeting the statistical bar (§16); 2. `to_tool`'s `args_model` is satisfiable from `from_tool`'s structured output, **possibly via a typed field remapping** (`args_binding`) — note this is *more* than today's detection, which validates the raw payload with **no remapping and no `from_tool` gating** (`react_runtime.py:179–218`). The application path is therefore net-new (§5.7, §8); 3. `to_tool` side-effect class is `pure` / `read` (write edges → human-only path) and `to_tool` is not a blocked node (`_MULTI_ACTION_BLOCKED_NODES`: `final_response`, `tasks.spawn`, rich-output; `react_runtime.py:61`); 4. across those traces the LLM never chose a *different* downstream tool given that output.

Emit `LearnedEdgeCandidate` (§9.3): tool names + typed field-to-field binding + **field-level bound-field pins** — a structural signature per **bound** field, so a tool change invalidates the binding only when it touches the shape of a field the binding actually uses; a version bump that leaves the bound fields intact does not retire the edge. The whole-model `args_model` schema-version pin is retained only as the conservative fallback where field-level signatures are unavailable (pinning on the full version string alone would silently bound edge lifetime by the tool-schema deploy cadence). No literal values stored.

5.5 (D) Validation gate (reuses PR #115 sweep)

1. Build the dataset (`TraceExampleV1` + manifest) from the **out-of-time** window (`export.py`), split val/test; the curated regression set is always in test.
2. Metric authority is **asset-specific** (§5.2). **Skills:** the positive signal is the `helpers.py` reference-guided goal-success score against the curated gold set, fed into the §16 statistical bar; the execution-truth helpers (`trajectory_subset_match`, `extract_terminal_node`, `sequence_match`) run only as **regression tripwires**, never to block a candidate for changing behavior. On slices with no gold answer a reference-free score may only act as a tripwire or route to human adjudication — it cannot positively promote. **Edges:** the execution-truth helpers are the primary authority (the edge must reproduce the known transition), plus the gate's sample outcome check. Iceberg label is a reported secondary for both. A deterministic **safety floor** (`safety_pass_rate`) can veto promotion regardless of score. Enforce **dataset⍰metric coupling** (manifest pins `metric.id/version/requirements`; runner fails fast on mismatch — a gap PR #115 leaves open).
3. `run_manual_sweep` (`sweep.py`): baseline vs candidate; the winner must pass the test-holdout regression gate **and** `min_test_score` **and** the statistical bar (§16: minimum sample size + bootstrap CI lower bound on the score delta > minimum detectable effect) **and** the

safety floor. For skills the score delta is on the **outcome/goal-success** metric over the gold slice; for edges it is on the fidelity/outcome metric over the matched cohort.

4. Emit an `ImprovementRun` (§9.4) and, for edges, a versioned `PatchBundleV1`.

5.6 (E1) Skill activation (small net-new write path)

The store **cannot persist learned scoped records today**: `upsert_pack_skill` hardcodes `origin="pack"`, writes scope `None,None`, and refuses non-pack rows (`skills/local_store.py:99–100,110,124`); and uniqueness is `name TEXT NOT NULL UNIQUE` — a single column declared **inline** (`local_store.py:455`), which blocks two tenants holding the same-named learned skill. v1 adds: - a **learned-skill write path** (`upsert_learned_skill`) writing `origin="learned"` + scope; - a **schema migration** that makes the identity key the full scope tuple **including** `scope_mode`. A plain `UNIQUE(name, scope_tenant_id, scope_project_id)` does **not** isolate scopes: the scope columns are nullable (`local_store.py:453–454`) and SQLite (today's only skill backend) treats `NULL` as DISTINCT inside a UNIQUE constraint, so two global skills `(NULL, NULL)` with the same `name` coexist and an upsert never matches them. Because the broken constraint is declared **inline on the column**, it cannot be dropped in place (SQLite has no `ALTER ... DROP CONSTRAINT`); the migration is therefore a **table rebuild** — create the new `skills` table *without* the inline `name UNIQUE`, copy rows, drop the old table, rename — and only then create the scoped-identity index that folds the global scope to a sentinel:

```
CREATE UNIQUE INDEX idx_skills_identity ON skills(
    name,
    scope_mode,
    COALESCE(scope_tenant_id, '__global__'),
    COALESCE(scope_project_id, '__global__')
);
```

The columns stay nullable, so the read-side scope filter is untouched — it still treats `NULL` scope columns as **scope wildcards** (`scope_mode='tenant' AND (scope_tenant_id IS NULL OR = ?)`, `provider.py:91–103`), not as the literal global scope; only the identity index folds `NULL → '__global__'`, making each `(name, scope_mode)` row unique. The sentinel is a reserved value the scope writer rejects as a real `tenant_id` / `project_id`, so a real id can never collide with it. The learned write path then upserts deterministically against the same tuple:

```
INSERT INTO skills (...) VALUES (...)
ON CONFLICT (name, scope_mode,
             COALESCE(scope_tenant_id, '__global__'),
             COALESCE(scope_project_id, '__global__'))
DO UPDATE SET ...;
```

An expression unique index is a valid `ON CONFLICT` target in SQLite; the same shape carries forward to the planned Postgres-backed store (which can instead use native `NULLS NOT DISTINCT`), but the current store is SQLite-only — there is no Postgres skill path to migrate today. - a **learned-store provider** fanned in via `CompositeSkillProvider`. Injection/formatting (`format_for_injection`) is reused unchanged, but the **read/dedup path is net-new**: today `get_by_name` collapses rows by `name` into `{row.name: skill}` (`local_store.py:248`) and the composite provider dedups first-fetched-wins by `name` (`provider.py:331–340`), neither of which applies scope precedence. Returning the in-scope row deterministically is net-new read-side work (next paragraph).

Because the identity key includes `scope_mode`, a global and a tenant skill may legitimately share a `name`, and the scope filter (`_build_scope_filter`, `provider.py:91–106`) returns **both** to an in-scope tenant with **no ordering**. The existing name-keyed read path cannot pick between them — both `get_by_name`'s name-collapse and `_dedupe_by_name` are first-fetched-wins, so retrieval would be decided by fetch order, not scope. v1 therefore adds a **net-new scope-aware dedup** to the read/inject path (§8) that applies **scope precedence — tenant > project > global** keyed on `name`, so a more specific scope shadows the broader one and exactly one record per `name` is injected. The names themselves are **never** scope-qualified: `skill.name` is emitted verbatim into the planner prompt (`_skill_block`, `provider.py`) and indexed into FTS, so encoding a raw `tenant_id` into the name would leak tenancy into the prompt and pollute retrieval ranking (§10) — scope stays in the columns + identity index, and precedence is resolved in the reader. This scope-precedence rule — most-specific scope (tenant > project > global) shadows the broader — is **shared by both learned asset classes**, differing only in the key it resolves on: skills by identity (`name`), edges by **decision point (`from_tool`)** so a tenant's edges for a step fully own that step (§5.7).

Light gate: structural validation + redaction (§10) + dedup + the holdout regression check from (D). Auto-activation only for advisory skills below a risk threshold; otherwise human approval → `active`.

5.7 (E2) Edge activation — learning IS the auto-seq authority

Design principle (architect): auto-learning implies auto-seq. A promoted learned edge grants auto-seq for its `from_tool → to_tool` transition directly — it does **not** require the developer to have set the static `auto_seq=True` / `auto_seq_execute=True` flags (`react_runtime.py:197`, `:1646–1661`). Those flags remain the *manual* path to auto-seq; learned edges are the *earned* path, where the validation gate (out-of-time holdout + CI, then shadow → canary with edge-outcome evidence entering at canary) is the evidence-based equivalent of a developer flipping the flag. Enabling the Learning Control Plane implies learned auto-seq is on; a `learned_auto_seq_enabled` knob exists for the niche "learn skills only" deployment.

“Implied” means opt-in is no longer required — not that the grant is unconditional. “Auto-seq” is two engine stages with two master switches: *detection eligibility* (per-tool `auto_seq`, `react_runtime.py:197`; master `_auto_seq_enabled`, L1646) and *execution eligibility* (per-tool `auto_seq_execute`, L1661; master `_auto_seq_execute`, L1659). **A learned edge replaces both per-tool flags; it never overrides the two operator master switches** (see precedence). A learned edge lives inside these boundaries:

- **Operator masters dominate.** If an operator set `_auto_seq_enabled=False` / `_auto_seq_execute=False` (the default), there is **no** model-bypass — learned included. `learned_auto_seq_enabled` cannot re-enable what the operator disabled.
- **Fire-time correctness, per instance** (net-new application logic): previous step == `from_tool`; the typed `args_binding` validates against `to_tool`’s `args_model` (pinned per bound field, §5.4); read-only is checked against the **live** `spec.side_effects` (not the stored class); and **ambiguity is scanned across the full live tool catalog** — not just `auto_seq=True` specs as static detection does (L197) — so a later-added tool that also matches the payload makes the edge defer to the LLM. Learning grants **eligibility**; the engine checks the **instance**.
- **Chain-depth bound.** Edges are mined and validated single-hop, but the planner re-runs detection every iteration (L1605), so learned hops can compose into a multi-hop chain that was never validated as a chain. v1 caps **consecutive learned-edge hops at 1** (default): after a learned edge fires, the next step returns to the LLM unless a multi-hop asset validated *as a chain* exists. The cap is configurable; >1 requires chain-level outcome validation.
- **ε-exploration at canary and active — randomize the fire decision.** After **all** fire-time checks pass, the engine fires the edge with probability 1−ε and defers to the LLM with probability ε (default ε = 0.05, configurable per scope; an operator may set 0). The coin outcome and its propensity are logged on the planner event. One mechanism buys three things: (1) **logged propensities** — the edge’s effect is estimated on a randomized contrast instead of matched-on-observables (§16); (2) a **permanent, in-scope control cohort** for rollback monitoring — the regression signal no longer depends on a pre-activation baseline that goes stale as the LLM drifts; (3) a **competition surface for skills** — on the ε-slice the LLM plans with learned skills injected, so a skill’s advice on an edge-owned step keeps producing signal instead of being starved by the edge always winning (resolves open question 5, §13). The coin runs *after* the checks, so the ε-slice measures edge-vs-LLM, never edge-vs-invalid.
- **Opt-out authoritative; opt-in not required — needs a NEW tool-author primitive.** Today `auto_seq` is tri-state-collapsed (`is not True`; unset ≡ `False`) with no opt-out, and `side_effects` defaults to the permissive `"pure"` (`catalog.py:78`). v1 adds (§8): (a) an explicit `never_auto_seq` opt-out marker, honored over any learned grant; (b) the eligibility default for an **unmarked** tool is **explicit read/pure side-effect AND not blocked AND not opted-out** — a tool left at the permissive `"pure"` default is *not* auto-eligible; autonomous activation requires the class to be set deliberately. `_MULTI_ACTION_BLOCKED_NODES` (`final_response`, `tasks.spawn`, rich-output; L61) stay off-limits.

- **Side-effect class is pinned.** The candidate's `side_effect_class` is versioned like the args_model pin; a reclassification (read→write or new instrumentation) invalidates the edge and forces re-validation, while fire-time reads the live `spec.side_effects` as authority. A *read* tool that triggers downstream writes is not catchable by class alone and is amplified by composition — hence the chain cap and the human gate for anything non-trivial.
- **Read-only implied freely; writes need a human.** Explicit `pure` / `read` reaches `active` autonomously; `write` requires human promotion in v1.
- **Reversible.** Per-asset rollback (outcome regression), TTL/decay, and the global kill-switch apply.

Control precedence (strict, descending) — mandated: 1. `KILL_SWITCH=True` → all learned assets off. 2. Operator `_auto_seq_enabled=False` / `_auto_seq_execute=False` → no model-bypass at all (dominates `learned_auto_seq_enabled`). 3. `learned_auto_seq_enabled=False` → learned edges inert; static path unaffected. 4. Per-edge `PromotionDecision.state` ∈ {`active`, or `canary` within cohort} → otherwise inert. Precedence is then resolved on the **decision point (`from_tool`)**, not the full transition: among in-scope learned edges leaving the same `from_tool`, only those from the **most-specific** scope (tenant > project > global, §5.6) survive — broader-scope edges from that `from_tool` are dropped **before** ambiguity detection, so a global `A → B` can never suppress a tenant `A → C` by inducing learned-edge ambiguity. A tenant that has learned *any* edge from `A` thus fully owns the step after `A` (v1 note: the miner emits only tenant-/project-scoped learned assets — §10 — so a *global* learned edge cannot arise in v1; the global tier of this rule covers a future, separately-justified cross-tenant aggregation path). If the winning scope still holds multiple edges from `A` to different `to_tool`s, ordinary fire-time ambiguity/validation applies among them (defer to the LLM if more than one validates). 5. Fire-time, in order: `never_auto_seq` opt-out → blocked nodes (L61) → binding pins (bound-field signatures; whole-model version as fallback, §5.4) → side-effect-class pin + live read-only check → full-catalog ambiguity defer → chain-depth cap → ε-exploration coin (fires 1–ε, defers ε; §5.7).

- **New machinery:** a **patch-point registry** (`penguiflow/planner/patch_points.py`) with one v1 learned key `auto_seq.learned_edges`; registering an edge there **grants both per-tool auto-seq flags for that transition** (no static flag needed), still subject to the precedence above. Planner application keys on `from_tool`, applies the typed `args_binding` before validation, and enforces the full-catalog ambiguity scan, the live side-effect read-only check, the bound-field + side-effect pins, the chain-depth cap, and — last — the ε-exploration coin (propensity logged on the planner event). A candidate that would *always* defer at fire time (a live-catalog conflict already exists at promotion) is **blocked at promotion** and surfaced as a learner metric, so an approval is never burned on an edge that can never fire.
- **Edge-outcome evidence enters at canary, not shadow:**
  - **shadow:** loaded but **log-only**; the edge never fires, so it yields only a **fire-time safety and eligibility check** (args_binding validates, the live read-only check holds, no live-catalog conflict), a **divergence rate** ("edge would fire / select X" vs the LLM's actual choice, `auto_seq_shadow_match` / `auto_seq_shadow_divergence`), and a **viable canary cohort**.

On a divergent trace the recorded outcome belongs to the path the LLM **actually took**, not the never-executed edge path, so shadow **cannot** estimate the edge's outcome and v1 does **not** claim it: shadow has no fire-time coin, hence no logged propensities, so off-policy estimation is unavailable there (the ε-coin introduces propensities only from canary on, §16). shadow → canary advances on fire-time safety + divergence rate + no live-catalog conflict + a viable cohort — never on a diverged-trace outcome.
    - **canary:** the edge **actually fires** for a scoped slice (tenant/project allowlist preferred over %), under the ε-exploration coin; this is the **first** stage that yields edge-outcome evidence — it compares the **observed outcome** of fired traces vs the **randomized ε-control** (coin-deferred traces in the same cohort/window) with a minimum sample and CI; a matched-control comparison is the fallback only while the ε-sample is inviable. canary → active advances on that observed randomized contrast.
    - **active:** full scope. **decay/rollback:** §16 defines the regression signal/threshold.
- Write-capable edges: machinery supported, but cannot reach `active` without explicit human promotion in v1.

5.8 (F) Governance

- **PromotionLedger** (append-only, versioned): every transition with `approved_by`, `improvement_run_id`, `scope`, `ttl`, `reason`.
- **Human approval** reuses Iceberg's HITL surfaces (`triage_queue` / contradiction / `privacy_review_task`). **Approval-throughput note:** every edge promotion and every write-edge is a triage item; v1 batches promotions per scope into a single review and sets a reviewer SLA, rather than one item per candidate.
- **Global kill-switch:** a single flag (per deployment, and per scope) that disables *all* learned assets at once, independent of per-asset rollback.
- **Decay/retirement:** typed half-lives (RFC_SKILLS_LEARNING_V213: browser 7d, api 90d, code 180d, domain 365d). Unused/regressing assets auto-retire.

5.9 (G) Learning-system observability

First-class metrics for the *learner itself*: precision of promoted assets (fraction that survive without rollback), false-promotion rate, % of production traffic served by learned edges, rollback frequency, mean time-to-promote, reviewer queue depth, and **never-fired edge rate** (edges blocked at promotion or that always defer at fire time). The skill track's gold economics are metrics too: **`gold_items_adjudicated` per cycle**, judge–gold agreement, and an explicit **`gate_inert (gold-starved)`** state — a cycle in which candidates existed but none could clear the §16 CI bar for lack of gold is *reported as such*, never answered by relaxing the bar. These gate whether the program is net-positive.

6. Control flow (one scheduled run)

```
# traces with neither an Iceberg row nor a scope_snapshot have no scope owner: they
never enter
# candidate mining (skills/edges) for any scope (global included); a separate non-
scoped pass folds
# them into global execution-truth/regression signals only (§4, §5.1).
spine.fold_unowned_into_global_signals(since=watermark)            # execution-
truth/regression only; no mining
for scope in scopes_due:
    traces  = spine.pull_labeled_traces(scope, since=watermark)   # (A)+(B),
incremental; scope-owned traces only
    skills  = skill_miner(traces)                                  # (C1)
    edges   = edge_miner(traces)                                   # (C2)
    dataset = build_oot_dataset(scope, window=W+1)                 # (D) out-of-time
+ curated set
    for cand in skills + edges:
        if not meets_support_bar(cand): continue                  # (§16) statistical
pre-gate
        run = validation_gate(cand, dataset, curated_set)          # (D) authority =
curated+exec metric
        if run.passed_with_ci:
            if cand.is_skill:  activate_skill(cand, run)           # (E1)
            else:              stage_edge_shadow(cand, run)        # (E2) shadow
first, always
        ledger.record(cand, run)                                   # (F)
    advance_promotions(scope)   # shadow→canary on divergence+fire-time safety;
canary→active on the observed ε-randomized contrast + approvals; rollback on
regression vs the concurrent ε-control
    emit_learner_metrics(scope) # (G)
    watermark.commit(scope)
```

7. Service ⍰ core boundary

In core (net-new unless noted)	In control-plane service
Contract types (§9)	Signal spine (A), label compiler (B)
Learned-skill write path + scoped-identity migration (E1)	Miners (C1/C2) with statistical bars

In core (net-new unless noted)	In control-plane service
Patch-point registry + planner edge application + ε-exploration coin & propensity logging (E2)	Validation orchestration over `penguiflow/evals` (D)
Global kill-switch; shadow/canary events	PromotionLedger + scheduler + Iceberg triage + learner metrics (F/G)
`SupportsTraceQuery` window enumeration; persisted `scope_snapshot` on `Trajectory`	—
Reused: skill injection/formatting (`format_for_injection`); auto-seq detection scaffolding; evals sweep	—

8. Build vs reuse (corrected)

Need	Reuse	Build
Execution truth	`SupportsTrajectories/PlannerEvents` (`get_trajectory`, `list_planner_events`)	**`SupportsTraceQuery` window enum** (existing `list_traces` needs `sessio` tenant scoping comes from the Iceber
Outcome truth	Iceberg `/knowledge`, `/facets`, `feedback`, `preference_score`	Label Compiler (B) + leakage firewall
trace⍰feedback join	Iceberg-driven set-extract on `agent_interaction_id == trace_id` (indexed where the column exists; §4)	startup env-capability check + index/b metadata fallback; tenancy reconciliati **persisted `scope_snapshot` on `Traj`**
Dataset+scoring+holdout	`penguiflow/evals` + `helpers.py` (PR #115)	OOT dataset builder; dataset⍰metric statistical/CI gating
Skill activation	skill **injection/formatting** (`format_for_injection`)	**learned-skill write path; scope-awar dedup with tenant>project>global p (the existing name-keyed read colla fetch order, not scope); scoped-iden uniqueness migration (table rebuild `name, scope_mode, COALESCE(scope_tenant_id,'__glob COALESCE(scope_project_id,'__glo` learned-store provider**

Need	Reuse	Build
Edge activation	auto-seq **detection scaffolding** (`DetectionResult` + auto-seq event types)	**patch-point registry; planner applica (from_tool keying + remap-then-vali binding);** `never_auto_seq` **opt-out n full-live-catalog ambiguity scan; bou side-effect-class pins; chain-depth shadow/canary; ε-exploration coin propensity logging on planner event**
Governance/HITL	Iceberg triage pattern	PromotionLedger; **global kill-switch**; metrics (G)

9. Contracts (sketch)

9.1 TraceOutcomeLabel

```
class TraceOutcomeLabel(BaseModel):
    trace_id: str                       # == agent_interaction_id
    tenant_id: str | None               # may be unknown if no Iceberg feedback row
    user_id: str | None
    session_id: str | None
    label: Literal["good", "bad", "mixed", "unknown"]
    confidence: float                   # [0,1]
    signals: dict[str, Any]             # intent, like, dislike, dwell_sec,
        exported, contested,
                                        # pref_delta, exec_error, tool_failures,
        replans
    source: Literal["iceberg", "curated", "merged"]
    compiled_at: datetime
```

9.2 LearnedSkillCandidate (origin="learned")

```
class LearnedSkillCandidate(BaseModel):
    skill: SkillRecord
        # origin="learned"; scope_mode + scope_* set (identity key)
    provenance_trace_ids: list[str]
    support_count: int                  # distinct good traces; gated by §16
    outcome_stats: dict[str, float]
    source_hash: str
    is_negative: bool = False
```

9.3 LearnedEdgeCandidate

```python
class LearnedEdgeCandidate(BaseModel):
    from_tool: str
    to_tool: str
    args_binding: dict[str, str]        # to_tool arg -> from_tool output field (no
        literals)
    bound_field_pins: dict[str, str]    # structural signature per bound field; a
        bound-field shape
                                        # change invalidates the edge — unrelated
        bumps do not (§5.4)
    to_tool_args_model_version: str     # whole-model fallback pin, used only where
        field-level
                                        # signatures are unavailable
    side_effect_class: Literal["pure", "read", "write"]
    side_effect_class_version: str      # pin; reclassification invalidates the edge
    max_learned_chain_hops: int = 1
        # consecutive learned hops allowed; >1 needs chain validation
    scope_mode: Literal["tenant", "project", "global"]   # parity with skills; v1
        miner emits tenant/project only (§10)
    scope_tenant_id: str | None
    scope_project_id: str | None
    provenance_trace_ids: list[str]
    support_count: int                  # gated by §16
    llm_choice_was_unambiguous: bool    # LLM never chose a different downstream
        tool (miner-side)
```

9.4 ImprovementRun & PromotionDecision

```python
class ImprovementRun(BaseModel):
    run_id: str
    asset_type: Literal["skill", "edge"]
    candidate_id: str
    dataset_id: str                    # out-of-time window
    metric_id: str                     # decision metric: gold goal-success signal (skill) | exec-truth fidelity (edge)
    baseline_score: float
    candidate_score: float
    score_delta_ci: tuple[float, float] # bootstrap CI; lower bound must exceed min effect
    n_val: int; n_test: int            # sample sizes (statistical bar)
    holdout_passed: bool
    safety_pass_rate: float            # deterministic safety floor; veto if below threshold (§5.5)
    safety_floor_passed: bool
    iceberg_label_corroboration: float  # secondary, reported only
    patch_bundle: dict | None          # PatchBundleV1 for edges
    created_at: datetime


class PromotionDecision(BaseModel):
    asset_id: str
    asset_type: Literal["skill", "edge"]
    version: int
    state:
        Literal["draft","validated","shadow","canary","active","retired","rolled_back"]
    scope_mode: Literal["tenant", "project", "global"]   # identity/precedence key
    scope_tenant_id: str | None
    scope_project_id: str | None
    exploration_epsilon: float | None   # canary/active fire-time coin (§5.7); None = scope default, 0 disables
    improvement_run_id: str
    approved_by: str | None            # None = automatic (advisory skills under risk threshold only)
    ttl: datetime | None
    reason: str
    decided_at: datetime
```

9.5 Patch-point registry

```
# penguiflow/planner/patch_points.py
class PatchPoint(BaseModel):
    key: str                              # v1: "auto_seq.learned_edges"
    schema: type[BaseModel]

        # Eligibility uses the LIVE spec.side_effects at fire time (authority), not
        this default.
    # Catalog default is the permissive "pure" (catalog.py:78), so autonomous
        activation requires
    # an EXPLICIT read/pure class — never the unset default (§5.7).
    side_effect_default: Literal["pure","read","write"] = "read"
    requires_human_for_write: bool = True


PATCH_POINTS: dict[str, PatchPoint]      # allowlist; unknown keys rejected
KILL_SWITCH: bool                        # precedence #1 — dominates all (§5.7)
LEARNED_AUTO_SEQ_ENABLED: bool           # precedence #3 — subordinate to operator
        _auto_seq_enabled
```

9.6 ScopeSnapshot (persisted on `Trajectory`)

```
class ScopeSnapshot(BaseModel):
    scope_mode: Literal["tenant", "project", "global"]
    tenant_id: str | None
    project_id: str | None
    user_id: str | None        # pseudonymized per §10; never compared against raw
        Iceberg user_id
```

`Trajectory.scope_snapshot: ScopeSnapshot | None`, captured at request start (§4) and carried on its own field with dedicated serialise/deserialise — never gated by whether the rest of `tool_context` is JSON-clean. The §4 reconciliation asserts agreement on `tenant_id` / `project_id` only; the snapshot's `user_id` is pseudonymized and stays out of the comparison.

10. Security & privacy (release-blocking)

- **Skills store abstractions only** — trigger/steps/preconditions/failure-modes; never raw tool observations, raw user content, or literal arg values. A secret/PII scrubber runs on candidate text; anything that still trips PII detection is dropped.
- **Edges store structure only** — tool names + typed field-to-field bindings; no literals.
- **PII in the mining substrate.** Mining reads raw `trajectory.steps`, and dataset rows are built from raw traces. Redaction must apply to **mining input and the persisted intermediate dataset**, not only stored asset text: the OOT dataset is built with the evals redaction profiles

(the relaxed `poc_full_context` is local-only and never persisted), and the spine drops fields not needed for mining before any persistence. The request-time `scope_snapshot` (§4) passes the same redaction profile before it is persisted on the trajectory: only the scalar scope fields are kept and `user_id` is pseudonymized.

- **Tenancy isolation.** Every learned asset is scoped from the reconciled tenancy (§4); cross-tenant mining is forbidden — consequently the **v1 miner never emits a `scope_mode="global"` learned asset**: the global tier in the contracts (§9.3/§9.4) and the precedence rules (§5.6/§5.7) exists for hand-authored packs and for a *future* cross-tenant aggregation path, which would need its own privacy argument (k-anonymity across tenants at minimum) before it may exist; activation filters by scope; the scoped-identity migration (§5.6) — a table rebuild that drops the inline `name UNIQUE` and adds a unique index over `(name, scope_mode, COALESCE(scope_tenant_id,'__global__'), COALESCE(scope_project_id,'__global__'))` — makes the store enforce *write-side* identity (one record per `(name, scope_mode, scope)`, including for global `(NULL, NULL)` rows that a plain multi-column UNIQUE would leave unconstrained under `NULL`-distinct semantics), while *read-side* isolation between same-named scopes is resolved by the scope-aware precedence reader (§5.6). The two together — not the index alone — enforce isolation.
- **Auditability.** Every promotion is in the PromotionLedger with provenance trace_ids; rollback is one ledger transition; the kill-switch is the blanket stop.

11. Rollout phases (v1)

1. **Substrate.** Land the evals stack (PR #115). A real trial-merge onto current `release/3.11` conflicted on **only `.DS_Store`** (source auto-merged), so the rebase is small, but Phase 0 also **adds** `SupportsTraceQuery` window enumeration and dataset⍰metric coupling — those are new work, not free. **Ordering rationale (v0.9 — edges before skills).** The edge track's authority signal — execution-truth fidelity plus the ε-randomized canary contrast on production traffic — is abundant and self-feeding; the skill track's authority — curated gold — is scarce and human-bounded. Blast radius is containable by machinery (ladder, precedence lattice, kill-switch); signal starvation is not containable by machinery, only by the gold workstream. v1 therefore ships edges first and starts gold curation early, so the skill track begins with gold in hand rather than sitting inert.

2. **Spine + labels — run as a feasibility experiment with a pre-registered go/no-go.** (A)+(B) + leakage firewall + watermark + tenancy reconciliation. Output: out-of-time, outcome-labeled datasets. No asset generation. Beyond proving the join + labels end-to-end, Phase 1 runs the loop **dry** and measures the three numbers the whole program depends on:

    - **signal density** — good-labeled traces per scope per week;
    - **candidate supply** — transitions/clusters clearing the §16 support bar over the dry window;
    - **gold throughput** — items/cycle actually adjudicated, vs the §16 power-analysis requirement.

Go/no-go (kill criteria fixed before any miner is built): Phase 2 starts only if edge-candidate supply clears its pre-registered floor; if it never does, the program stops here — the labeled datasets remain standalone value. If gold throughput falls short, Phase 3 is **deferred** while the gold workstream catches up; the CI bar is never relaxed to compensate (a starved cycle is reported as `gate_inert`, §5.9). The **gold curation workstream starts in this phase** — harvesting failure/repair cases from the OOT windows into human-adjudicated gold — as a staffed, budgeted activity, not a side effect.

3. **Edge track (read-only).** Patch-point registry + planner application + shadow → canary → active with ε-exploration, edge-outcome evidence entering at canary (E2). Read-only edges only reach `active` autonomously (still gated statistically + monitored); write edges require human promotion. Runs entirely on the self-feeding signal while gold accrues.

4. **Skill track — gated on the gold floor.** Learned-skill write path + migration + miner (C1) → gate (D, with CI over the curated gold) → activation (E1) → governance (F-light) + learner metrics (G). Low blast radius but gold-bounded: it starts when accrued gold clears the §16 sample floor for at least one slice — not before, and never by lowering the bar.

12. Testing strategy

- **Label compiler:** synthetic Iceberg feedback → expected labels; curated-override precedence; tenant/project-disagreement drop; a missing `scope_snapshot` *with* an Iceberg row still mines/labels on Iceberg tenancy (no drop) and is excluded only from the canary cohort; a trace with **neither** an Iceberg row **nor** a `scope_snapshot` is excluded from scoped mining and feeds only global execution-truth/regression signals.
- **Miners:** golden trajectories → expected candidates; dedup; ambiguity/blocked-node rejection; support-threshold enforcement.
- **Gate:** reuse `tests/evals`; add tests that a regressing candidate fails; that a candidate with too few samples or CI lower-bound ≤ min effect is rejected; that a skill is decided by the gold outcome signal (an Iceberg-label-only "win" does not promote, and a behavior-changing skill that improves the gold outcome is **not** blocked by tripped imitation metrics); that an edge is decided by fidelity + the sample outcome check; and that the safety floor vetoes regardless of score.
- **Edge application:** the learned edge fires only when previous step == `from_tool`, the binding validates, and ambiguity defers to the LLM (parity with `react_runtime` detection tests); shadow mode never changes behavior; a bound-field-pin (or fallback version-pin) mismatch disables the edge, while an unrelated arg-model bump that leaves the bound fields intact does not.
- **ε-exploration:** the coin runs only after every fire-time check passes (the ε-slice measures edge-vs-LLM, never edge-vs-invalid); deferral + propensity are logged on the planner event; ε=0 disables exploration for a scope; skills are injected on the ε-slice; canary statistics use the randomized contrast and fall back to matched controls only under an inviable ε-sample.

- **Promotion:** state-machine tests; shadow→canary advances on divergence + fire-time safety + a viable cohort (never on a diverged-trace outcome); canary→active advances on the observed ε-randomized contrast; rollback on injected outcome regression vs the concurrent ε-control; scope isolation; kill-switch disables everything.
- **Live probe:** controlled end-to-end on one prod agent (shadow only) before any canary.

13. Open questions

1. Support thresholds N, min sample sizes, and minimum detectable effect — **pre-registered as the Phase 1 go/no-go criteria (§11)**: seeded conservatively before the dry loop, held fixed for the go/no-go read, then tuned via (G).
2. Canary cohorting: tenant/persona allowlist (preferred) vs percentage-of-traffic.
3. Control-plane service home (own repo vs `penguiflow-admin` adjacent).
4. Negative-skill injection ergonomics (consume "don't do this" without context bloat).
5. **Conflict resolution** (skill advises X, edge bypasses to Y on the same step) — **largely resolved in v0.9 by ε-exploration (§5.7)**: an active edge owns 1−ε of its step, never all of it, and on the ε-slice the LLM plans with skills injected, so the skill keeps generating comparable signal instead of being starved. Remaining sub-question: the action when the ε-slice shows the skill-advised LLM path beating the edge — proposed: that *is* the §16 regression signal, so it triggers edge rollback + a triage item.
6. Cold start: until support accrues, agents rely on hand-authored skill packs (existing); the loop produces nothing for brand-new agents/tenants — acceptable for v1, flagged.

14. Alternatives considered

- **One unified asset/loop** (original proposal): rejected — different blast radii need different gates.
- **PatchBundle as the skill container:** rejected — skills use the store path; only edges need patch machinery.
- **Iceberg labels as the gate authority:** rejected — that signal is used to mine candidates; using it to also judge them is leakage (§5.2).
- **Online/in-request learning:** rejected — unbounded blast radius; industry pattern (DSPy/GEPA, LangSmith) is offline mine → eval → gate → canary.
- **Matched-control-only canary (no randomization):** rejected — the fire decision is platform-controlled, so an ε-holdout with logged propensities is nearly free and upgrades the edge effect estimate from selection-on-observables to a randomized contrast (§5.7/§16).
- **New governance UI:** rejected — reuse Iceberg triage.

15. Code anchors

Anchors on `release/3.11` HEAD are **verified against the working tree**; anchors marked *(PR #115)* are verified against the open PR branch (a real trial-merge onto `release/3.11` conflicted only on `.DS_Store`) and line numbers may drift when it lands.

- Skills: `penguiflow/skills/provider.py` (`SkillProvider` L35, `LocalSkillProvider` L343, `CompositeSkillProvider` L788, tenant filter L96); `penguiflow/skills/models.py` (`SkillRecord` L129, `origin` L12); `penguiflow/skills/local_store.py` (`upsert_pack_skill` origin/scope/refusal L99/L110/L124, `name UNIQUE` L455).
- Auto-seq: `penguiflow/planner/react_runtime.py` (`DetectionResult` L172, `_detect_deterministic_transition` L179, `auto_seq` gate L197, blocked nodes L61, execute gates L1646-1661).
- StateStore: `penguiflow/state/protocol.py` (`StateStore` L16, `SupportsTrajectories` L111/L114/L116, `SupportsPlannerEvents` L120); `penguiflow/state/models.py` (`RemoteBinding` tenant/user L60-62); production Postgres impl in `test_generation/pengui_canvas/.../stores/state/postgres.py`.
- Evals *(PR #115)*: `penguiflow/evals/` (`export.py` `TraceExampleV1`, `sweep.py` `PatchBundleV1` + `run_manual_sweep`, `helpers.py` metric helpers, `api.py` metric decorator, `__pf_patch_bundle` convention); `SupportsTraceQuery` / `TraceRef` added to `state/protocol.py`; `wait_for_trace_persistence` in `react_runtime.py`.
- Iceberg: `interaction_fragment` (`src/pengui_iceberg/persistence/sql_stores.py:360`, `agent_interaction_id` unique index `idx_interaction_agent_id_unique`); column-capability probe `_ensure_agent_interaction_id_capability` L1473, `metadata ->> 'agent_interaction_id'` Postgres fallback L1972 (unindexed; SQLite cannot resolve the join when the column is absent); ingest schema `src/pengui_iceberg/memory/schemas.py:72`, feedback L42; `knowledge_fragment` / `facet` / `preference_score`; HITL `triage_queue` / `contradiction` (`migrations/.../021_metacognition_*`); idempotency precedent `knowledge_watermark`.

16. Statistical validity, leakage & promotion signals

This section specifies the methodology that makes promotion decisions trustworthy.

- **Support threshold.** A candidate needs ≥ N distinct *good* traces (not occurrences) from ≥ M distinct users, to resist single-user/self-selected bias. Seed conservatively; tune via (G).
- **Sparsity reality.** Explicit Iceberg feedback is a small, self-selected fraction of traffic (failures over-reported), so it stays a mining/ranking signal, not the authority. **Authority is asymmetric:** skills are decided by a gold-calibrated outcome/goal-success signal over the curated gold set (feeding the CI bar below), with the execution-truth metrics (node sequence/terminal node) reserved as regression tripwires; edges are decided by those same execution-truth (fidelity) metrics plus a sample outcome check, since reproducing the known transition is the edge's

contract. Outcome decides whether a *skill* is better; imitation/fidelity metrics guard regressions and certify *edges*.

- **No point-score promotion.** The gate computes a **bootstrap confidence interval** on the candidate–baseline score delta on the test holdout; promotion requires the **CI lower bound > minimum detectable effect** and `n_test ≥` a floor — not a bare `min_test_score`.
- **Out-of-time holdout.** Mine on `W`, validate on `W+1` (§5.2) to break train/test contamination.
- **Leakage firewall.** The **mining signal** (Iceberg feedback) is excluded from the gate decision — not all outcome evaluation. Decision = curated gold-set outcome signal (skills) / execution-truth fidelity + sample outcome check (edges), with a judge model family distinct from any candidate generator, gold human-adjudicated **blind** to the mined signal, and gold items firewalled out of the miner for the cycle they validate (§5.2).
- **Provable improvement is bounded by gold coverage.** The curated gold set is the only fully independent positive signal for skills, and it is scarce: a skill's improvement is *proven* only on the slice the gold covers; behavior change outside that slice is unproven, not validated, and a reference-free judge there may only tripwire or route to human review — never promote. The program grows the bound deliberately — each cycle harvests new failure/repair cases from the out-of-time window into curated gold (human-adjudicated, and excluded from the miner for any cycle in which they serve as authority). Gold is periodically recalibrated and contamination-checked as the agent and models evolve, so frozen gold does not silently drift out of distribution. Where gold alone cannot clear the CI bar, PPI may supplement it — under the conditions in the dedicated bullet below.
- **Gold power analysis — what the CI bar actually costs.** For a binary goal-success metric near p ≈ 0.7 with MDE = 5pp at 80% power / α = 0.05, an unpaired two-arm comparison needs ≈ 1,300 gold items per arm; the gate's paired design (baseline and candidate scored on the *same* gold items) cuts that roughly 3–5× under typical agreement rates — i.e. **order low hundreds of adjudicated gold items per slice, per cycle** (MDE = 10pp needs ~4× fewer; halving the MDE quadruples the bill). These are the skill track's real unit economics: v1 treats **gold curation as a staffed, budgeted workstream starting in Phase 1** (§11), tracks `gold_items_adjudicated`/cycle in (G), and reports a cycle that cannot clear the bar as `gate_inert (gold-starved)` (§5.9) — the answer to scarcity is deferral plus more gold, never a lower bar.
- **PPI — conditions, realistic gain, degradation.** Prediction-powered inference (the PPI++ / power-tuned-λ variant) widens the effective sample by combining scarce gold with abundant judge labels through a bias correction estimated on the gold. It is valid only under **all** of:
  1. the judge emits **paired per-item predictions** on every gold item *and* on the unlabeled slice, from the same judge version/prompt; (2) the gold is a **representative, task-signature-stratified sample** of the eval slice, refreshed as the traffic mix shifts; (3) the bias-correction term is re-estimated each cycle (judge drift breaks a frozen correction). The gain is bounded by judge–gold correlation — realistically a **1.3–2× effective-sample multiplier, not 10×** — so PPI softens the gold bill, it does not repeal it. Degradation rule: if judge–gold agreement on the cycle's fresh gold falls below a floor, the gate reverts to gold-only CI for that cycle and reports it.

- **Edge-outcome evidence enters at canary — as a randomized contrast.** Canary advances on the **observed outcome** of fired traces vs the **ε-control** (coin-deferred traces with logged propensities, §5.7), with sample minimums; matched controls are the fallback only while the ε-sample is inviable. Shadow advances on divergence rate + fire-time safety + cohort viability, **never** on diverged-trace outcomes: the outcome on a divergent trace belongs to the path the LLM took, and shadow has no coin and hence no propensities (the coin exists only once the edge can fire), so off-policy estimation of the edge's counterfactual outcome is unavailable there and is not claimed.
- **Rollback signal.** Auto-rollback when, over a defined window and minimum sample, an active asset's cohort shows an outcome-rate drop beyond a threshold versus its **concurrent ε-control** (preferred: a live randomized comparison, immune to LLM/model drift) or, where the operator set ε = 0, its pre-activation baseline. Divergence-from-LLM is monitored but is not, alone, a rollback trigger (the LLM drifts over time).
- **Confounding — mostly retired in v0.9.** The fire decision is platform-controlled, so canary and active **randomize it** (ε-exploration, §5.7): the edge's effect estimate is a randomized contrast with logged propensities (IPS/DR estimators apply), not selection-on-observables. Residual accepted limits: a scope with ε = 0 falls back to matched controls (covariates: task signature, tenant, model version, time window — selection-on-observables at best); shadow yields no edge-outcome evidence at all; and write edges never auto-promote.

17. Revision history

- **v0.9:** (1) **Inverted Phases 2/3 — edges before skills** (§11): the edge track's authority (execution-truth fidelity + the randomized canary contrast) is abundant and self-feeding, while the skill track is gold-bounded; blast radius is containable by machinery, signal starvation is not. The skill track is **retained, gated on the gold floor** — not descoped. (2) Made **Phase 1 an explicit feasibility experiment** with pre-registered go/no-go criteria (signal density, candidate supply, gold throughput) and kill criteria fixed before any miner is built; a gold-starved cycle is reported as `gate_inert` (§5.9), never answered by relaxing the CI bar; gold curation becomes a staffed workstream from Phase 1. (3) Added **ε-exploration** at canary/active (§5.7/§16): a fire-time coin after all checks with logged propensities — yielding a randomized effect contrast (largely retiring the matched-control confounding caveat), a permanent drift-immune control cohort for rollback, and a competition surface that resolves the skill-starvation half of open question 5.
    1. Added the **gold power analysis** (order low hundreds of paired items per slice at MDE = 5pp) and a full **PPI conditions/gain/degradation** spec (§16). (5) Resolved the **global-scope contradiction**: the v1 miner emits only tenant-/project-scoped learned assets (§10); the global tier in contracts and precedence covers hand-authored packs and a future, separately-justified cross-tenant aggregation path. (6) Replaced the whole-model args_model version pin with **field-level bound-field pins** (whole-model pin retained as fallback), so unrelated tool-schema bumps no longer retire edges (§5.4/§9.3).

- **v0.8:** Resolved edge precedence on the **decision point (`from_tool`)** rather than the full transition: when in-scope edges leave the same `from_tool`, only the most-specific scope's edges survive, dropped **before** ambiguity detection — so a broader global edge can no longer suppress a narrower tenant edge (e.g. global `A → B` vs tenant `A → C`) by inducing learned-edge ambiguity. A tenant that has learned any edge from a step owns that step (§5.6/§5.7).
- **v0.7:** Gave **learned edges** the same scope-precedence rule as skills (tenant > project > global; most-specific shadows broader, one edge per transition), stated once as a shared cross-asset rule (§5.6/§5.7). Made positive mining require an Iceberg outcome signal — a trace with no Iceberg row is `unknown` (execution-truth/regression and negative-skill mining only, never "good", §5.2). Distinguished the two rollout ladders in the overview's top-level loop diagram.
- **v0.6:** (1) **Read-path:** learned skill names are **not** scope-qualified — `skill.name` reaches the planner prompt and FTS, so a `tenant_id` in the name would leak tenancy (§10); the name-keyed read path (`get_by_name`, `_dedupe_by_name`) collapses same-named scoped rows by fetch order, not scope precedence, so v1 adds a **net-new scope-aware read/dedup** with tenant>project>global precedence while the scoped-identity migration stays the write-side isolation mechanism. Relabeled the skill read path from "reused" to net-new across §1/§5.6/§7/§8 and deleted the claim that the existing read path applies scope precedence. (2) **Shadow yields no edge outcome:** a log-only edge never fires, so a divergent trace's outcome belongs to the LLM's path (no logged propensities → off-policy estimation unavailable); edge-outcome evidence now enters only at **canary**, with shadow advancing on divergence + fire-time safety + cohort viability. Updated §1, §3, §5.2, §5.7, §6, §11, §12, §16, and the diagram. Also: §4 tenancy agreement is **tenant/project only** (pseudonymized `user_id` excluded); traces with neither an Iceberg row nor a `scope_snapshot` are excluded from scoped mining and feed only global signals (non-scoped fold in §6); narrowed the auto-seq reuse claim to detection scaffolding; and added `ScopeSnapshot` (§9.6), `safety_pass_rate` / `safety_floor_passed` (§9.4), and `scope_mode` on `LearnedEdgeCandidate` (§9.3) and `PromotionDecision` (§9.4).
- **v0.5:** Hardened persistence reality, the trace⍰feedback join, and gate authority. (1) Replaced best-effort `tool_context` tenancy with a first-class persisted, sanitized `scope_snapshot` on `Trajectory` (`serialise()` nulls `tool_context` on any non-JSON value, `trajectory.py:245–251`): offline mining/labels stay on authoritative Iceberg tenancy, a missing snapshot no longer drops a trace, and the snapshot gates only request-time canary admission. (2) Made the trace⍰feedback join **Iceberg-driven** (batch set-extract per `(tenant, window)`) with a startup capability check and index/backfill where the deployment is on the `metadata ->> 'agent_interaction_id'` fallback. (3) Replaced the single-column inline `name UNIQUE` (`local_store.py:455`) with a table rebuild plus a scoped-identity unique index over `(name, scope_mode, COALESCE(scope_tenant_id,'__global__'), COALESCE(scope_project_id,'__global__'))` and a deterministic upsert, so same-named global skills no longer collide under `NULL`-distinct semantics. (4) Split gate authority by asset class: skills are decided by a gold-calibrated goal-success signal feeding the statistical bar (with a safety floor), imitation metrics demoted to regression tripwires; edges keep fidelity as

the primary authority with a sample outcome check; gold is human-adjudicated blind and firewalled out of the miner for the cycle it validates.

- **v0.4:** Bounded the “auto-learning implies auto-seq” grant — added the control-precedence lattice (kill-switch > operator masters > `learned_auto_seq_enabled` > per-edge state > fire-time checks), the consecutive-learned-hop cap (default 1), the `never_auto_seq` opt-out marker with a conservative default for unmarked tools, full-live-catalog ambiguity scanning, side-effect-class pinning, and the block-at-promotion guard for never-firing edges.
- **v0.3:** Adopted “auto-learning implies auto-seq”: a promoted edge grants auto-seq directly; the static `auto_seq_execute` flag is no longer a precondition, with the grant kept bounded (opt-out authoritative, operator masters dominate, reversible).
- **v0.2:** Hardened methodology — leakage firewall, out-of-time holdout, outcome-based shadow/canary, statistical/CI gating, tenancy reconciliation, kill-switch, and PII handling in the mining substrate; marked the net-new core work (skill write path, skill-store uniqueness migration, `SupportsTraceQuery` enumeration, edge-application path) explicitly.
- **v0.1:** Initial draft.